2.4 Automatyzacja powtarzalnych zadań: Tworzenie własnych skryptów wspieranych przez AI do migracji danych i boilerplate'u

Moduł 2: Ekosystem Vibe Coding i AI-first IDE

Krzysztof Rytlewski · 2026

Automatyzacja wspierana przez AI – zmiana paradygmatu

Tradycyjnie developer samodzielnie pisze skrypty do powtarzalnych zadań: migracje bazy danych, generowanie boilerplate'u, transformacje plików. To praca czasochłonna i rzadko dobrze udokumentowana.

Modele językowe zmieniają ten paradygmat: opisujesz zadanie w języku naturalnym i iteracyjnie doprecyzowujesz wynik – AI pełni rolę doświadczonego kolegi znającego składnię Bash, Node.js i cloud CLI na pamięć.

Kluczowe typy zadań do automatyzacji z AI:

Kategoria	Przykłady
Migracje danych	Zmiana schematu MongoDB, transformacja CSV/JSON
Boilerplate	Nowy moduł Angular, nowy serwis Express
Transformacje plików	Batch rename, konwersja formatów, parsowanie logów
Zadania DevOps	Skrypty deploymentu, backup, czyszczenie zasobów AWS

Struktura dobrego prompta do generowania skryptu

Surowy prompt *„napisz mi skrypt do migracji MongoDB"* przyniesie generyczny wynik. Precyzyjny opis przyniesie skrypt gotowy do uruchomienia.

Pięć elementów dobrego prompta:

1. **Cel** – co skrypt ma osiągnąć (jedno zdanie).
2. **Dane wejściowe** – format, lokalizacja, przykładowy rekord.
3. **Dane wyjściowe** – oczekiwany wynik, format, lokalizacja.
4. **Ograniczenia** – timeout, obsługa błędów, idempotentność.
5. **Technologia** – język, biblioteki, wersje.

Technika iteracyjna przy tworzeniu skryptów

Zacznij od wersji podstawowej, uruchom na zbiorze testowym, a następnie iteruj z modelem, dodając kolejne wymagania:

- *„dodaj progress bar dla dużych kolekcji"*

- *„obsłuż przypadek gdy kolekcja jest pusta"*

- *„dodaj tryb dry-run który tylko raportuje co zostałoby zmienione"*

Każda iteracja jest małym krokiem – model zachowuje kontekst poprzednich decyzji i dodaje tylko żądaną funkcjonalność bez przepisywania całego skryptu.

Migracje MongoDB – Typ 1: Dodanie pola z wartością domyślną

```
// scripts/migrations/001-add-permissions-to-admins.js
async function up() {
  const db = mongoose.connection.db;
  const result = await db.collection('users').updateMany(
    { role: 'admin', permissions: { $exists: false } },
    { $set: { permissions: ['read', 'write', 'delete'] } }
  );
  console.log(`Zaktualizowano: ${result.modifiedCount} dokumentów`);
}

async function down() {
  await db.collection('users').updateMany(
    { role: 'admin' },
    { $unset: { permissions: '' } }
  );
}
```

Migracje MongoDB – Typ 2 i 3

Typ 2 – Zmiana nazwy pola (bezpieczna, dwuetapowa):

```
// Krok 1: skopiuj wartość do nowego pola
await db.collection('orders').updateMany(
  { totalPrice: { $exists: true } },
  [{ set: { totalAmount: 'totalPrice' } }]
);
// Krok 2 (po weryfikacji): usuń stare pole
await db.collection('orders').updateMany({}, { $unset: { totalPrice: '' } });
```

Typ 3 – Transformacja wartości:

```
// Normalizacja emaili do lowercase
await db.collection('users').updateMany(
  {},
  [{ set: { email: {toLower: '$email' } } }]
);
```

Konwencja numerowania i runner migracji

Numeruj pliki migracji sekwencyjnie: `001-opis.js`, `002-opis.js` – zapewnia deterministyczną kolejność wykonania (analogia do Flyway/Liquibase).

Rejestr wykonanych migracji w MongoDB:

```
// Sprawdź czy migracja była już wykonana
const executed = await db.collection('_migrations')
  .findOne({ name: '001-add-permissions' });
if (executed) {
  console.log('Migracja już wykonana, pomijam.');
  process.exit(0);
}
// ... wykonaj migrację ...
await db.collection('_migrations').insertOne({
  name: '001-add-permissions',
  executedAt: new Date()
});
```

Generowanie boilerplate'u – podejście przez prompt

Podejście 1 – prompt z wzorcem: Przekaż istniejący plik jako wzorzec i poproś o stworzenie analogicznego dla nowej domeny:

> *Stwórz moduł `products` analogiczny do istniejącego modułu `users`. Pliki: `product.model.js`, `product.service.js`, `product.routes.js`, `product.service.test.js`. Wzorzec w `src/users/`. Zamiast `User` używaj `Product`, zamiast `/api/users` używaj `/api/products`.*

Wynik jest spójny z istniejącą architekturą, bo model widzi prawdziwy wzorzec – nie generuje w próżni.

Generowanie boilerplate'u – własny skrypt generatora

Podejście 2 – skrypt generatora (sam napisany z pomocą AI):

```
// scripts/generate-module.js
const resourceName = process.argv[2]; // np. "product"
const ResourceName = resourceName[0].toUpperCase() + resourceName.slice(1);

const targetDir = path.join('src', `${resourceName}s`);
fs.mkdirSync(targetDir, { recursive: true });
// ... zapis szablonów plików ...
```

```
node scripts/generate-module.js product
# Tworzy: src/products/product.model.js, product.service.js, itd.
```

Podejście 3 – Aider nieinteraktywnie z flagą `--message`: przekazuje prompt do Aidra bez sesji dialogowej, gotowe do użycia w skryptach CI/CD.

Automatyzacja zadań AWS z pomocą AI i AWS CLI

AI znacząco obniża barierę wejścia do AWS CLI, generując złożone komendy z opisu naturalnego:

```
# Opis: "skrypt czyszczący pliki S3 starsze niż 90 dni"
aws s3api list-objects-v2 \
  --bucket backups-prod \
  --query "Contents[?LastModified<='$(date -d '90 days ago' --iso-8601)'].Key" \
  --output text | \
  xargs -I {} aws s3 rm s3://backups-prod/{}
```

Integracja z Open Interpreter – dla zadań eksploracyjnych: `Sprawdź jakie mam aktualnie uruchomione instancje EC2 w regionie eu-west-1, pokaż ich nazwy, typy i miesięczne koszty szacunkowe`. Open Interpreter połączy się z AWS CLI i przedstawi wyniki.

Idempotentność – kluczowa własność skryptów automatyzacji

Skrypt idempotentny uruchomiony wielokrotnie daje ten sam wynik co uruchomiony raz – kluczowe przy migracjach.

```
// NIE idempotentne - za każdym razem dodaje duplikat
await db.collection('config').insertOne({ key: 'default_role', value: 'user' });

// Idempotentne - upsert nie duplikuje
await db.collection('config').updateOne(
  { key: 'default_role' },
  { $setOnInsert: { value: 'user' } },
  { upsert: true }
);
```

Tryb dry-run – bezpieczne testowanie skryptów

```
const DRY_RUN = process.env.DRY_RUN === 'true';

const docs = await db.collection('users')
  .find({ role: 'admin', permissions: { $exists: false } }).toArray();
console.log(`Znaleziono ${docs.length} dokumentów do aktualizacji`);

if (!DRY_RUN) {
  await db.collection('users').updateMany(...);
  console.log('Aktualizacja wykonana.');
} else {
  console.log('[DRY-RUN] Aktualizacja pominięta.');
}
```

```
DRY_RUN=true node scripts/migrations/001-add-permissions.js
node scripts/migrations/001-add-permissions.js
```

Tryb dry-run pozwala zweryfikować zakres zmian bez faktycznej modyfikacji danych produkcyjnych.